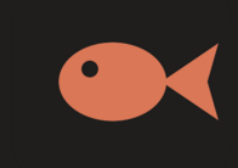
ANDROID 16 · COLOROS 16 · LIVE UPDATES



工作岛

把 Claude 的工作状态，放到灵动岛上

宿主	cccodroid 0.4.2
设备	OnePlus PJA110 · ColorOS 16.0.3
应用	com.dafeiyu.workisland 1.0
状态	测试阶段 · 仅 ColorOS 16 / Android 16 验证
日期	2026-10-25

APK SHA-256
79697f3779e590412dc5eca43b948018f40d6f61f6ee1cddb725e8f85da2140e

目录

这份材料按“为什么做 → 怎么设计 → 怎么实现 → 怎么验证 → 怎么提交”组织。

文档	内容
测试状态	仅在 ColorOS 16 / Android 16 验证；未验证项、已知限制、反馈方式
项目说明	一句话、交付物、快速开始、测试环境
一、方案与设计	背景与问题、总体架构、状态机、事件文案、收工判定、关键技术、安全与限制
二、开发与部署	目录落点、构建链、安装、网关接入、driver 部署、保活、调试、回滚
三、测试报告	10 个用例、关键日志、截图索引、6 个问题与修复
四、提交说明	验收步骤、设计要点、已知限制、变更记录

> **三个非显然结论**
>
> 1. ColorOS 16 的流体云直接接 Android 16 Live Updates，不需要 OPPO 私有白名单。
> 2. ColorOS 会在后台冻结应用（连前台服务也冻），所以定时器必须放在网关 driver 里。
> 3. 收工要看 `runs` 而不是 `run`，并且要二次复查 + 3 秒宽限，否则会提前报完成。

测试状态

> **当前状态：测试阶段（Pre-release / Alpha）**
>
> **目前只在一种设备组合上测试成功： OnePlus PJA110 · ColorOS 16.0.3 · Android 16（SDK 36）· cccodroid 0.4.2。**

项目仍在测试阶段：接口、通知文案、网关接入方式和文件布局都可能继续变动，**不建议用于生产或关键任务**。

已验证

- ColorOS 16 流体云能接住 Android 16 Live Update（`PROMOTED_ONGOING` 实测出现）
- 工作态「正在干活」、完成态「任务完成」、8 秒后收起
- 并行轮次结束不会误报完成
- driver 被杀后由守护循环/网关看门狗拉起
- cccodroid 与工作岛都退到后台 8 秒，driver、网关、前台服务仍存活
- 构建、签名、安装、通知授权在设备上通过

未验证

- 其他 ColorOS 版本（如 15、17）与其他机型
- 其他厂商的“灵动岛 / 实时活动”实现（HyperOS、OriginOS、One UI、MagicOS 等）
- 开发端网关（8790）与 cccodroid（8791）的差异场景
- 重启开机自启（`BOOT_COMPLETED` 已声明，未做整机重启验证）
- 长时间 Doze（30 分钟以上）
- force-stop 后的自动恢复
- 多轮并行时的文案归属（收工判定已按 `runs` 处理，文案可能跟随最后事件）

已知限制

- cccodroid 被强行停止后，网关与 driver 都会消失，岛上会停在最后状态；重开 cccodroid 恢复。
- ColorOS 会在后台冻结应用（`do_freezer_trap`，前台服务也冻），所以定时器放在网关 driver，而不是应用里。
- cccodroid 升级可能覆盖 `files/app` 下的网关文件，需要重新接入。
- 工作岛的服务是 exported 的，同机应用可以伪造显示状态（仅影响显示）；生产化需加签名级权限。
- 胶囊外观由系统绘制，未使用 OPPO UPK 自定义卡片。

反馈

如果你在别的 ColorOS 版本或机型上试了，欢迎带上：

- 机型、ColorOS 版本、Android 版本
- `dumpsys notification --noredact | grep -A1 "pkg=com.dafeiyu.workisland"` 的 flags 行
- 现象截图

这些信息足够判断是“没推广”“被冻结”还是“收工判定”的问题。

项目说明

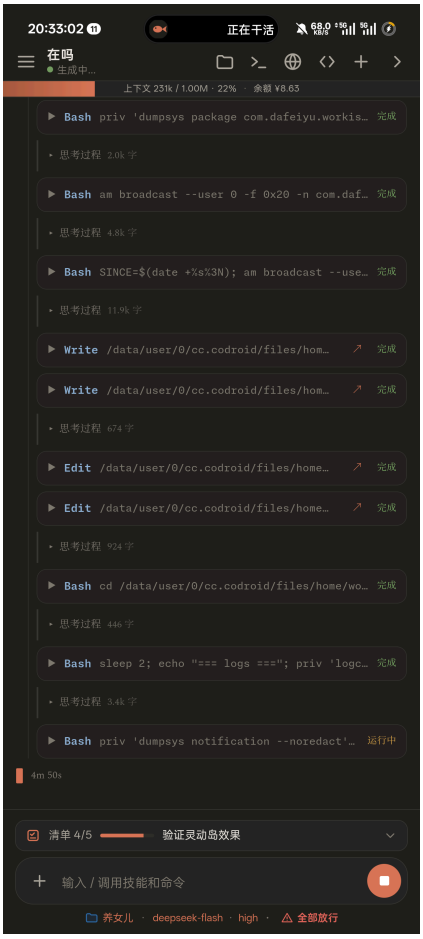
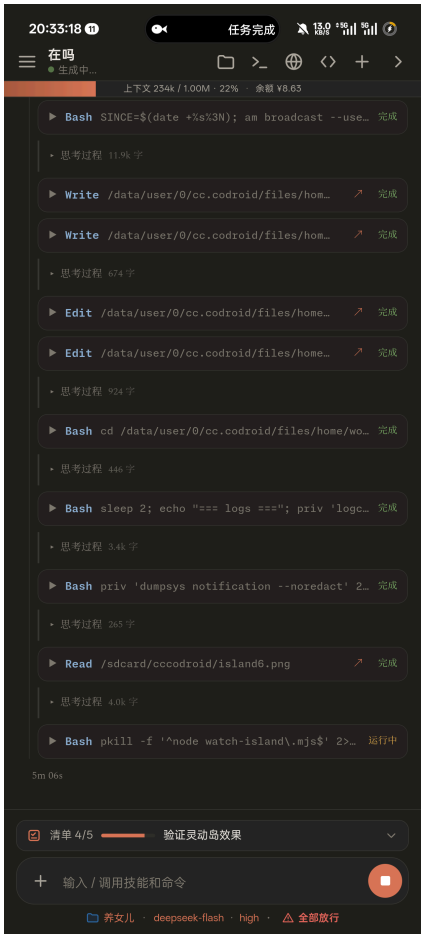
工作岛 · WorkIsland

> **测试状态：Pre-release / Alpha 目前只在 OnePlus PJA110 · ColorOS 16.0.3 · Android 16（SDK 36）· cccodroid 0.4.2–0.4.6 上测试成功。** 其他 ColorOS 版本、其他品牌机型均未验证；接口与文件布局可能继续变动，不建议用于生产或关键任务。 详细边界见 `STATUS.md` 。

把 **cccodroid 里 Claude 的实时工作状态**，投到 **ColorOS 16 的流体云 / 灵动岛**上。

- 干活时：岛上写「正在干活」，下面跟着「用 执行命令…」「正在想…」「等你确认…」
- 收工：写「任务完成」+ 用时
- 8 秒后：岛上收起，通知栏只留一条安静的「工作岛 · 在后台等下一轮」

不是另开一个任务清单，也不是模拟动画——数据直接来自 cccodroid 网关的真实事件流。

正在干活	任务完成	8 秒后收起
		

更多截图见 `screenshots/`，完整边界见 `STATUS.md`，方案见 `docs/01-方案与设计.md`。

下载：v1.1.0 发布页 · APK 与 PDF 在 Release 附件里。

交付物

路径	内容
`apk/WorkIsland.apk`	可直接安装的 APK（`com.dafeiyu.workisland`，versionName 1.1）
`source/app/`	Android 16 Live Update 应用源码（Java + 资源 + 构建脚本）
`source/driver/driver.mjs`	独立状态驱动：0.4.2–0.4.6 的部署方式
`source/gateway/work-island.mjs`	0.4.2–0.4.6 的网关模块（spawn 独立 driver）
`source/gateway/server.mjs`	0.4.2–0.4.6 已接入的网关入口（原文件见 `server.mjs.original`）
`source/upstream-pr-review.patch`	0.4.7 上游 PR #20 的完整 diff（网关内订阅 + 设置 + 单测）
`screenshots/`	工作态、完成态、收起、后台存活、应用界面截图
`docs/`	方案与设计、开发部署、测试报告、提交说明

APK 校验：

```
SHA-256  728c4f3a8ff020ea9c9ad3cdac7ffa4c235c7c41030868aa77e1628cc9d681e6
大小     41,929 字节
```

快速开始

1. 安装 `apk/WorkIsland.apk`。
2. 打开「工作岛」，确认三行状态： - 通知：开 - 实时更新：允许 - 电池：不优化
3. 接入网关： - **ccc odroid 0.4.7+**：设置 → 工作岛（ColorOS 灵动岛）→ 打开，重启一次网关。 - **0.4.2–0.4.6**：按 `docs/02-开发与部署.md` 第 4 节接入独立 driver。
4. 在 cccodroid 里让 Claude 干一次活，看岛。

测试环境

项	值
设备	OnePlus PJA110
系统	ColorOS 16.0.3（Android 16 / SDK 36）
cccodroid	0.4.2
网关	127.0.0.1:8791
应用	com.dafeiyu.workisland 1.1
构建	aapt2 2.20-android-16 / d8 9.2.4 / apksigner 0.9 / javac 21
编译 SDK	Android 16 platform-36（android.jar）

一句话设计

> **网关负责“知道”，模块负责“判断”，工作岛负责“画”。** 定时器和事件流不放在会被 ColorOS 冻结的应用里，放在网关进程里。

0.4.7 起上游改为进程内订阅（`onPublish` + `runList`），不再单独起 driver；本仓库的 `source/driver/` 是 0.4.2–0.4.6 的部署方式。两种方式发送给同一个应用。

详细设计见 `docs/01-方案与设计.md`。

许可与第三方说明

本项目使用 **Apache-2.0**，见 `LICENSE`。

> 工作岛是**第三方应用的可选集成**：cccodroid 只负责把状态送出来，不包含也不负责本应用的功能。

工作岛 · 方案与设计

> 版本 1.1 · 2026-10-25 设备：OnePlus PJA110 · ColorOS 16.0.3（Android 16 / SDK 36） 宿主：cccodroid 0.4.2（包名 `cc.codroid`）

摘要

「工作岛」把 cccodroid 里 Claude 的实时运行状态，显示在 ColorOS 16 的流体云（灵动岛）上：

- 进行中：**正在干活** + 当前动作（用执行命令 / 正在想 / 等你确认…）
- 完成：**任务完成** + 用时
- 8 秒后：岛上收起，通知栏保留一条安静的「在后台等下一轮」

数据不是模拟的，来自 cccodroid 网关的真实事件流（`/api/events`、`/api/state`）。整个方案不修改 cccodroid 的 Java 代码，只做两处可回退的网关接入。

1. 背景与问题

cccodroid 是运行在 Android 上的 Claude Code 宿主：它有一个 Node 网关（`server.mjs`，监听 `127.0.0.1:8791`），负责会话、运行队列、事件流和通知。

原生通知是普通通知，不会自动变成 ColorOS 的流体云胶囊。要上岛，需要 Android 16 的 **Live Updates**（实时更新）资格：

- 通知使用 `Notification.ProgressStyle` 等标准样式；
- 声明 `POST_PROMOTED_NOTIFICATIONS` 权限；
- 通知为 ongoing，带标题，不使用自定义视图；
- 通过 `android.requestPromotedOngoing` 请求推广（promoted）。

在实现过程中遇到并解决了四个真问题：

问题	现象	根因
普通通知不上岛	只有通知栏，没有胶囊	没有 promoted / ProgressStyle
完成后不收起	岛上一直挂着「正在干活」	监听器跟着单轮 Claude 进程被收走
定时器不生效	「任务完成」永不收起	ColorOS 把应用冻住（`do_freezer_trap`），进程内定时器不走
提前报完成	还在干活就显示「任务完成」	把「最近一轮」当成「所有轮次」，换轮空隙 `run` 短暂为空

2. 目标与非目标

目标

1. 岛上显示 **真实** 的 Claude 工作状态（阶段 / 工具 / 完成），不是写死动画。
2. 视觉与 cccodroid 统一：深色 `#1F1E1D`、陶土色 `#D97757`、鱼形图标。
3. 后台可用：cccodroid 切后台、工作岛切后台、屏幕关闭时仍能更新。
4. 收工准确：只有所有运行轮次都结束才报完成；连续两轮之间不闪「完成」。
5. 可回退：对 cccodroid 的改动只有两个文件，原文件有备份。

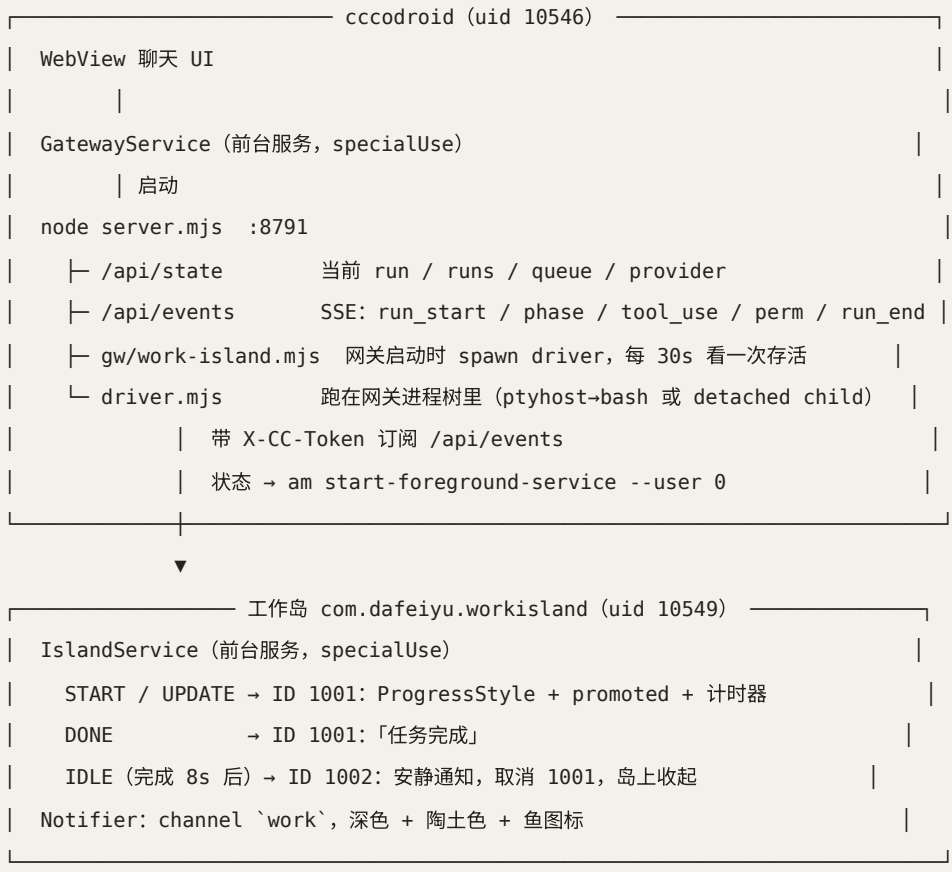
非目标

- 不接管 cccodroid 的聊天 UI、不修改它的 Java/ dex。
- 不做到「ccc odroid 被强行停止后仍然工作」——没有宿主就没有任务可显示。
- 不自定义胶囊的像素级外观（胶囊由系统绘制，第三方只能控制图标、标题、文本、进度）。
- 不承诺跨设备通用：当前实现针对 Android 16 / ColorOS 16。

---

3. 总体架构

三层，职责单一：



一句话：

> **网关负责“知道”，模块负责“判断”，工作岛负责“画”。**

> 0.4.7 起上游采用进程内订阅（`onPublish` + `runList`），本仓库的独立 driver 是 0.4.2–0.4.6 的部署方式；两者发送给同一个应用。

---

4. 运行流程

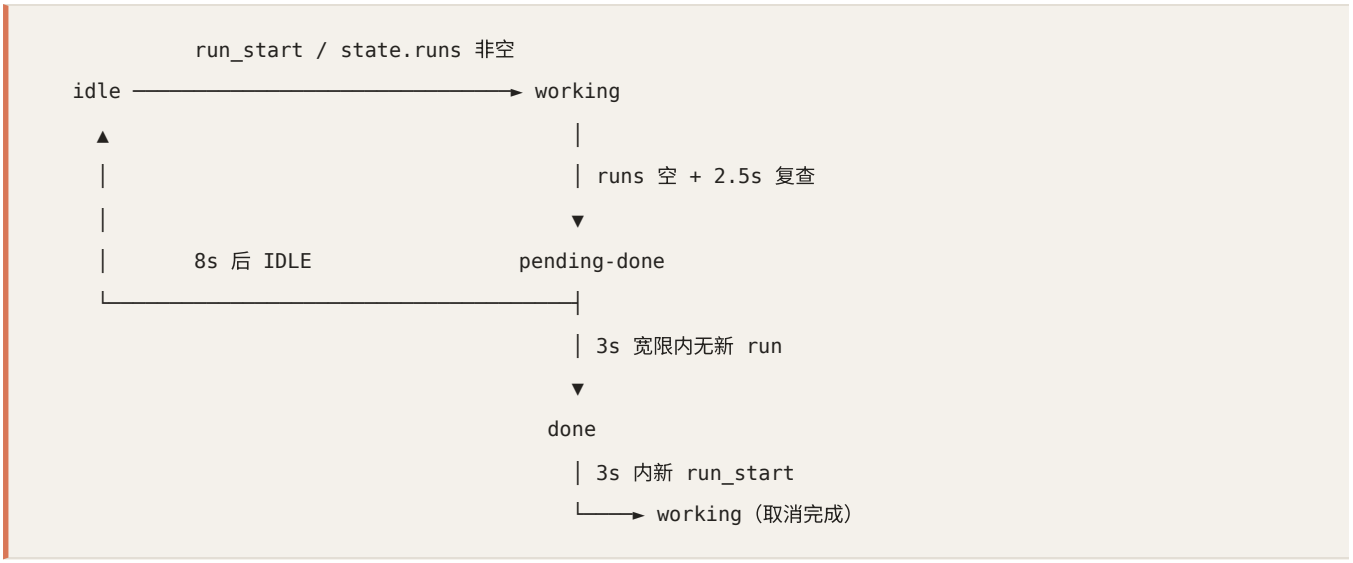
4.1 首次接入

1. 安装 `WorkIsland.apk`，授予通知权限。
2. 系统设置里开启「允许实时更新」（`canPostPromotedNotifications()` 为 true）。
3. 网关启动时执行 `server.mjs` 里的 `startWorkIsland()`：- 读取 `~/.work-island.pid`，若对应进程仍活着就复用；- 否则 `spawn(process.execPath, [driver.mjs])`，`detached + unref`，并删除环境里的 `CCAPP_RUN`（避免被网关的孤儿清理误杀）。
4. driver 读取 `~/.gateway-token`（0600，仅 cccodroid uid 可读），订阅 `http://127.0.0.1:8791/api/events`。
5. driver 通过 `am start-foreground-service --user 0` 驱动工作岛；工作岛拉起前台服务并画通知。

4.2 一轮任务

```
用户发消息
  → 网关 run_start
  → driver 置 working，发 UPDATE「正在干活 / 正在处理…」
  → 模型思考 / 调工具：phase=think / tool / tool_use
  → 岛上实时更新「正在想…」「用 执行命令…」
  → 权限确认：perm_request → 「等你确认」
  → 回答后：perm_resolved → 「继续…」
  → run_end
  → driver 不立刻报完成，先等 2.5s 回 /api/state 复查
      ├─ runs 还有 → 说明是别的轮次结束，忽略
      └─ runs 为空 → 进入 3s 宽限
            ├─ 3s 内来了新 run_start → 取消完成，接着干
            └─ 没有新 run → 发 DONE「任务完成 / 用时 X」
                  └─ 8s 后发 IDLE → 换成 ID 1002 安静通知，岛上收起
```

4.3 状态机



5. 事件到文案

driver 把网关事件翻译成岛上的短句（尽量 6 个字以内，避免胶囊截断）：

事件	岛上文本
`run_start`	正在处理…
`phase=wait`	等模型回复…
`phase=think`	正在想…
`phase=text`	正在写回复…
`phase=tool` / `tool_use`	用 执行命令… / 读文件… / 改文件… / 搜代码… / 查资料…
`perm_request`	等你确认：<工具>
`perm_resolved`	继续…
`compacting`	整理上下文…
`sub_progress`	子任务：<描述>…
`run_idle` 且 `bg>0`	后台任务还在跑…
`error`	出错了：<前 50 字>
`run_end`	进入收工判定

工具名映射：`Bash→执行命令`、`Read→读文件`、`Write/Edit→写/改文件`、`Grep→搜代码`、`WebSearch→查资料`、`Task/Agent→子任务`、`TodoWrite→整理清单` 等。

标题固定为「正在干活」；权限确认时临时用「等你确认」；完成时用「任务完成」。

6. 收工判定：四道关

这是整个项目踩坑最多的地方，单独说明。

1. **看 `runs`，不看 `run`** `/api/state` 里 `run` 是“最近开始的一轮”，换轮的空隙可能短暂为空；`runs` 才是“所有正在跑的轮次”。只要 `runs` 非空，就不算完。
2. **`run_end` 后 2.5 秒复查** 收到 `run_end` 不直接发完成，先 `setTimeout 2500ms`，再 GET `/api/state`；`runs` 为空才继续。
3. **不按 `runId` 直接排除** 换轮时 `active.id` 可能还是上一轮的，按 id 排除会漏掉真正的结束；统一用第 2 步的状态复查。
4. **完成前 3 秒宽限** 复查通过后再等 3 秒；期间来了新 `run_start` 就取消完成，继续「正在干活」。这避免了连续两轮之间闪一下「任务完成」。

完成之后还有 8 秒的展示期，然后 IDLE 收起；新的一轮开始会取消 IDLE 定时器。

---

7. 关键技术

7.1 Android 16 Live Updates 与 ColorOS 16

ColorOS 16 的流体云已经接入 Google 的 Android 16 Live Updates 规范。满足资格的通知会被系统提升为胶囊，无需 OPPO 的私有白名单；OPPO 自家的 UPK 卡片是另一条路，需要业务资格。

资格要求（实测）：

- 使用标准样式（本项目用 `Notification.ProgressStyle`）；
- 声明 `android.permission.POST_PROMOTED_NOTIFICATIONS`；
- `setOngoing(true)`，有 `contentTitle`；
- 不使用 `customContentView`，不是 group summary，未 colorized；
- 通知渠道重要性不是 `IMPORTANCE_MIN`（本项目用 `IMPORTANCE_LOW`）；
- 请求 promoted（见 7.2）。

7.2 推广（promoted）的开启方式

平台公开的 `Notification.Builder` 没有 `setRequestPromotedOngoing` 方法，但系统里有：

- 额外字段键：`android.requestPromotedOngoing`（布尔）；
- 隐藏方法：`Notification.Builder#setRequestPromotedOngoing(boolean)`。

实现同时做两件事：

```
Bundle extras = new Bundle();
extras.putBoolean("android.requestPromotedOngoing", true);
builder.addExtras(extras);
try {
    Notification.Builder.class
        .getMethod("setRequestPromotedOngoing", boolean.class)
        .invoke(builder, true);
} catch (Throwable ignored) { /* 失败也不影响普通通知 */ }
```

验证方式：

```
dumpsys notification --noredact | grep -A1 "pkg=com.dafeiyu.workisland"
# flags=ONGOING_EVENT|ONLY_ALERT_ONCE|NO_CLEAR|FOREGROUND_SERVICE|PROMOTED_ONGOING
```

7.3 通知设计

项	进行中 / 完成	待机
通知 ID	1001	1002
样式	`ProgressStyle`（indeterminate / 100%）	普通通知
promoted	是	否
计时	`setUsesChronometer(true)`	无
小图标	鱼形矢量图	同
颜色	`#D97757`	无
标题	正在干活 / 任务完成	工作岛
文本	当前动作	在后台等下一轮

为什么待机要换 ID： 同一个 ID 的 FGS 通知在更新时可能保留 `PROMOTED_ONGOING` 标志，导致岛上不收起；换到 1002、取消 1001，胶囊就干净地消失。

为什么用 `ProgressStyle` 而不是普通进度条： 普通 `setProgress` 不满足 Live Update 的样式资格；`ProgressStyle` 才能进入流体云。

7.4 ColorOS 的后台冻结

实测发现：即使应用有前台服务、通知是 promoted，ColorOS 仍会在后台把进程冻住：

```
/proc/<pid>/task/<tid>/wchan = do_freezer_trap
```

后果：应用里的 `Handler.postDelayed`、`Thread.sleep`、SSE 读取都停住，8 秒收起永远不会发生。

结论：定时器和事件流不能放在工作岛应用里。 它们被搬到 driver，driver 跑在 cccodroid 网关的进程树里；网关是用户正在使用的宿主，不会被冻。工作岛退化成一个“只负责画”的渲染器，每次 `am` intent 把它短暂唤醒，画完继续冻着也没关系。

7.5 进程树与生命周期

driver 的两种部署方式：

1. **网关模块 spawn（生产）**：`gw/work-island.mjs` 在网关启动时 `spawn`，`detached + unref`，父进程是网关；网关每 30 秒检查一次 PID 文件，挂了就重启。
2. **网关终端里跑（调试/兜底）**：`POST /api/term/new` 建终端，输入 `while true; do node driver.mjs; sleep 2; done`； 进程链是 `node server.mjs → ptyhost → bash → node driver.mjs`，同样不属于单轮 Claude。

两种方式通过 `~/.work-island.pid` 去重：PID 对应进程的 cmdline 含 `driver.mjs` 就复用。

7.6 `am` 的限制

从应用 uid 直接 `am broadcast` 会失败：

```
SecurityException: broadcast asks to run as user -2 but is calling from uid u0aXXXX;
requires android.permission.INTERACT_ACROSS_USERS_FULL
```

解决：

- 显式加 `--user 0`；
- 改用 `am start-foreground-service --user 0 -n ...`，既绕开跨用户限制，又能把被冻结的工作岛唤醒；
- 失败重试两次（间隔 0.8 秒），覆盖“刚装完 APK、服务还没起来”的瞬时错误。

7.7 构建链（设备上无 Android SDK）

设备上只有 aapt2 / d8 / apksigner / javac，没有 Android SDK、没有 Gradle、没有 zipalign。构建步骤：

1. 下载 `platform-36_r02.zip`，取出 `android-36/android.jar`（编译用，27 MB）。
2. `aapt2 compile --dir res -o res.zip`
3. `aapt2 link -I android.jar --manifest AndroidManifest.xml --java gen -o base.apk res.zip`
4. `javac -source 11 -target 11 -classpath android.jar -d classes ...`
5. `d8 --lib android.jar --min-api 36 --output dex classes.jar`
6. `zip -j base.apk dex/classes.dex`
7. `apksigner sign --ks island.keystore ...`

`resources.arsc` 由 aapt2 产出时已经是未压缩且 4 字节对齐（实测 `data offset % 4 == 0`），因此没有 zipalign 也能安装。签名用自签名证书；重新签名会让已安装应用需要卸载重装。

7.8 保活

工作岛侧：

- 前台服务（`specialUse`）+ `android:stopWithTask="false"`：从最近任务划掉不停止服务；
- `BOOT_COMPLETED` 接收器：开机后恢复待机通知；
- `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`：界面上的「后台保活设置」按钮；
- 已加入 `deviceidle` 白名单，standby bucket 设为 active。

cccodroid 侧：

- 网关本身是前台服务；
- 已加入 `deviceidle` 白名单；
- ColorOS 自己的「自启动」「最近任务锁定」需要用户手动开启（adb 管不到）。

边界： 强行停止（force-stop）会杀掉一切，任何应用都无法自救；重新打开 cccodroid 后网关模块会重新拉起 driver。这是 Android 的规则，不是缺陷。

---

8. 接口与契约

8.1 网关 HTTP

接口	方法	说明
`/api/state`	GET	`run`（最近一轮）、`runs`（所有活跃轮次）、`queue`、`provider`、`model`
`/api/events`	GET (SSE)	`hello`、`run_start`、`state`、`phase`、`tool_use`、`perm_request`、`perm_resolved`、`run_idle`、`error`、`run_end`

鉴权：请求头 `X-CC-Token: <64 位十六进制>`，令牌在 `~/.gateway-token`（0600）。

8.2 工作岛 Service Intent

组件：`com.dafeiyu.workisland/.IslandService`

Action	Extra	行为
`...START` / `...UPDATE`	`title`, `text`, `since` (long, ms)	画进行中通知（ID 1001）
`...DONE`	无	画「任务完成」，等 driver 发 IDLE
`...IDLE`	无	换 ID 1002 安静通知，取消 1001
`...CANCEL`	无	取消通知并停止服务

启动方式：

```
am start-foreground-service --user 0 \
  -n com.dafeiyu.workisland/.IslandService \
  -a com.dafeiyu.workisland.START \
  --es title "正在干活" --es text "用 执行命令…" \
  --el since 1792932556566
```

---

9. 安全与隐私

- 网关只监听 `127.0.0.1`，并且所有 `/api/*` 需要令牌；令牌文件 0600，只有 cccodroid uid 能读。
- driver 以 cccodroid uid 运行，读取令牌，但 **不把令牌写进命令行、日志或工作岛**；工作岛只接收 intent。
- 工作岛的 `IslandService` 是 exported 的，同机其他应用可以伪造 START/DONE（只影响显示，不影响数据）。生产化时应加 signature 级自定义权限，或改为只允许同 uid/同签名调用。
- 不采集、不上传任何屏幕、位置、剪贴板或会话内容；岛上只显示当前动作的短句。
- 对 cccodroid 的改动可回退：`server.mjs.original` 是原文件，`server.mjs` 只多了一行 import 和一行启动调用。

---

10. 已知限制

1. **强行停止**：ccc odroid 被 force-stop 后，网关和 driver 都没了，岛上会停在最后一个状态；重开 cccodroid 恢复。

2. **ColorOS 冻结**：工作岛在后台会被冻，不能自己计时；本方案把计时放在 driver，但如果 driver 也被硬杀，收起可能延迟。
3. **应用更新**：ccc odroid 更新可能覆盖 `files/app/gw/work-island.mjs` 与 `server.mjs`，需要重新接入（原文件有备份）。
4. **胶囊外观**：由系统绘制，只能控制图标、标题、文本、进度；不能自定义成 OPPO UPK 那种完全自定义卡片。
5. **exported 服务**：见第 9 节，属于演示级安全。
6. **单实例**：driver 只跟踪一个“当前状态”；并行多轮时，文案可能跟随最后触发的事件，但收工判定仍以 `runs` 为准。

---

11. 后续演进

- 给工作岛 service 加签名级权限，杜绝伪造 intent。
- 在 driver 里加心跳（每 15–30 秒），配合 AlarmManager 看门狗，缩短硬杀后的残留时间。
- 把 driver 的路径/端口做成配置，支持开发端 8790 与 cccodroid 8791。
- 增加可选的胶囊文案模板（例如显示会话标题、token 用量）。
- 如果 OPPO 开放 UPK 资格，可做完全自定义的胶囊卡片。
- 把网关模块做成可安装组件（跟随 cccodroid 更新自动重放），减少手工接入。

工作岛 · 开发与部署

> 适用：Android 16 / ColorOS 16（OnePlus PJA110 实测） 宿主：cccodroid 0.4.2

1. 目录说明

交付包

```
工作岛-方案与设计/
├── README.md
├── docs/
│   ├── 01-方案与设计.md
│   ├── 02-开发与部署.md
│   ├── 03-测试报告.md
│   └── 04-提交说明.md
├── source/
│   ├── app/                  Android 应用源码（Java + res + build.sh）
│   ├── driver/driver.mjs     状态驱动
│   ├── driver/watch-island.mjs  早期独立 watcher（历史对照，不参与正式流程）
│   └── gateway/
│       ├── work-island.mjs         网关模块（启动 driver + 30s 看门狗）
│       ├── server.mjs              已接入的网关入口
│       └── server.mjs.original     接入前原文件
├── screenshots/
└── apk/WorkIsland.apk
```

设备上的落点

内容	路径
应用源码工程	`/data/user/0/cc.codroid/files/home/workspace/task-island/app/`
driver	`/data/user/0/cc.codroid/files/home/workspace/task-island/driver.mjs`
网关模块	`/data/user/0/cc.codroid/files/app/gw/work-island.mjs`
网关入口	`/data/user/0/cc.codroid/files/app/server.mjs`
网关原文件备份	`/data/user/0/cc.codroid/files/app/server.mjs.bak-workisland`
网关令牌	`/data/user/0/cc.codroid/files/home/.gateway-token`（0600）
driver PID	`/data/user/0/cc.codroid/files/home/.work-island.pid`

> `/data/user/0/cc.codroid/...` 是 cccodroid 的应用私有目录，只有 cccodroid uid 可读写；不要把它放到 `/sdcard` 上执行。

---

2. 环境与工具

设备上需要：

工具	实测版本	用途
aapt2	2.20-android-16.0.0_r4	资源编译与链接
javac	21	编译 Java
d8	9.2.4	生成 classes.dex
apksigner	0.9	签名
keytool	21	生成签名密钥
zip / unzip	—	打包
node	26	运行 driver 与网关
magick	—	生成启动图标（仅开发用）

没有 Android SDK、Gradle、zipalign；构建走 `aapt2 + javac + d8 + zip + apksigner`。

---

3. App 构建

3.1 取 android.jar

```
mkdir -p /data/user/0/cc.codroid/files/home/workspace/task-island/sdk36
curl -L -o platform-36.zip \
  https://dl.google.com/android/repository/platform-36_r02.zip
unzip -o platform-36.zip 'android-36/android.jar' \
  -d /data/user/0/cc.codroid/files/home/workspace/task-island/sdk36
# 结果：.../sdk36/android-36/android.jar（约 27 MB）
```

`build.sh` 默认使用 `../sdk36/android-36/android.jar`；如果你的目录不同，改脚本第一行的 `SDK=`。

3.2 构建

```
cd /data/user/0/cc.codroid/files/home/workspace/task-island/app
bash build.sh
# 产物：build/base.apk（未签名）
```

`build.sh` 做的事：

```
aapt2 compile --dir res -o build/res.zip
aapt2 link -o build/base.apk -I "$SDK" \
  --manifest AndroidManifest.xml --java gen --auto-add-overlay build/res.zip
javac -source 11 -target 11 -classpath "$SDK" -d build/classes @build/sources.txt
jar cf build/classes.jar -C build/classes .
d8 --release --lib "$SDK" --min-api 36 --output build/dex build/classes.jar
cd build && zip -q -j base.apk dex/classes.dex
```

3.3 签名

```
cd /data/user/0/cc.codroid/files/home/workspace/task-island/app

# 如果还没有密钥：
keytool -genkeypair -keystore island.keystore -alias island \
  -keyalg RSA -keysize 2048 -validity 10000 \
  -storepass island123 -keypass island123 \
  -dname "CN=WorkIsland, O=WorkIsland, C=CN"

apksigner sign --ks island.keystore \
  --ks-pass pass:island123 --key-pass pass:island123 \
  --out WorkIsland.apk build/base.apk
```

说明：

- 交付包**不包含私钥**（`island.keystore` 留在本机工作区）。重新生成密钥会改变签名，已安装的旧版必须先卸载再装。
- `resources.arsc` 由 aapt2 输出时已是未压缩且 4 字节对齐，实测可直接安装；如果换工具链后安装报 `resources.arsc ... not aligned`，再补 zipalign。
- APK 版本：`versionCode=2`、`versionName=1.1`。

3.4 安装

共享目录 `/sdcard` 里的 APK 不能直接 `pm install`（SELinux 文件上下文问题），先经 shell 拷到 `/data/local/tmp`：

```
cp WorkIsland.apk /sdcard/cccodroid/WorkIsland.apk
priv 'cp /sdcard/cccodroid/WorkIsland.apk /data/local/tmp/WorkIsland.apk && \
      chmod 644 /data/local/tmp/WorkIsland.apk && \
      pm install -r /data/local/tmp/WorkIsland.apk'
priv 'pm grant com.dafeiyu.workisland android.permission.POST_NOTIFICATIONS'
```

首次打开「工作岛」，确认：

- 通知：开
- 实时更新：允许
- 电池：不优化（点「后台保活设置」按提示开）

4. 网关接入

4.1 放入模块

```
cp source/gateway/work-island.mjs \
  /data/user/0/cc.codroid/files/app/gw/work-island.mjs
```

4.2 修改 server.mjs（两行）

先备份：

```
cp /data/user/0/cc.codroid/files/app/server.mjs \
   /data/user/0/cc.codroid/files/app/server.mjs.bak-workisland
```

在 import 区加：

```
import { startWorkIsland } from "./gw/work-island.mjs";
```

在 `server.listen(...)` 的回调末尾加：

```
try { startWorkIsland(); } catch (e) { log("工作岛启动失败: " + e.message); }
```

完整 diff 只有两处；`source/gateway/server.mjs` 是改好的版本，可直接对照。

4.3 生效

网关在 cccodroid 启动时加载 `server.mjs`。要让改动生效：

- 重启 cccodroid（或让它重启网关服务）。
- 重启会结束当前会话的生成，但会话记录还在，重新打开即可。

生效后网关日志会出现：

```
[work-island] driver 已启动 pid=...
```

4.4 不重启网关的临时接法（调试用）

通过网关的终端 API 直接跑 driver：

```
TOKEN=$(cat /data/user/0/cc.codroid/files/home/.gateway-token)

# 1) 建一个终端
R=$(curl -s -X POST -H "X-CC-Token: $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"rows":24,"cols":80,"title":"工作岛"}' \
  http://127.0.0.1:8791/api/term/new)
ID=$(echo "$R" | sed -n 's/.*"id":"\([a-z0-9]*\)".*/\1/p')

# 2) 在终端里跑一个守护循环
curl -s -X POST -H "X-CC-Token: $TOKEN" \
  --data-binary $'cd /data/user/0/cc.codroid/files/home/workspace/task-island && \
while true; do node driver.mjs; echo "driver exited, restart"; sleep 2; done\n' \
  "http://127.0.0.1:8791/api/term/$ID/in"

# 3) 看输出（SSE）
curl -s -N -H "X-CC-Token: $TOKEN" \
  "http://127.0.0.1:8791/api/term/$ID/stream"
```

终端进程链是 `node server.mjs → ptyhost → bash → node driver.mjs`，不属于单轮 Claude，因此不会被单轮收尾带走。

4.5 去重

- driver 启动时把 PID 写进 `~/.work-island.pid`。
- 网关模块启动时读这个文件，检查对应进程的 cmdline 是否包含 `driver.mjs`；是就复用，不重复拉起。
- 模块每 30 秒检查一次，driver 挂了会自动重启。

4.6 cccodroid 0.4.7+：上游网关内集成

0.4.7 起上游把工作岛改成了网关进程内订阅（PR #20），不再需要本仓库的独立 driver、PID 文件、令牌和看门狗：

- 网关模块：`gateway/gw/work-island.mjs` + `gateway/gw/work-island-core.mjs`；
- `gateway/gw/bus.mjs` 提供 `onPublish(fn)`；收工判定直接读 `runList()`；
- 设置 → **工作岛（ColorOS 灵动岛）** → 打开；默认关闭；
- 打开后才异步检测应用、开始送状态；关掉发一次 IDLE；
- 更新限频 1.5 秒；`error` 只显示「出错了」；文案跟随 `LAST_LANG`（中/英）；
- 清单 `<queries>` 已加 `com.dafeiyu.workisland`；如果设备上仍检测不到，可用 `CCAPP_WORK_ISLAND=1` 强制开启。

对本仓库的部署来说，0.4.7+ **不需要**第 4.1–4.5 节的独立 driver；保留它们是给 0.4.2–0.4.6 的。

---

5. driver 行为说明

`driver.mjs` 的关键逻辑：

```
读 ~/.gateway-token
  → SSE 订阅 /api/events
  → run_start / hello.run / state.run → 开始
  → phase / tool_use / perm_request → 更新文案
  → run_end → 等 2.5s → GET /api/state → runs 为空?
       否 → 忽略（别的轮次）
       是 → 等 3s → 有新 run_start?
              是 → 取消完成
              否 → 发 DONE → 8s → 发 IDLE
  → 每 5s 轮询 /api/state 兜底
  → SIGTERM / SIGINT / SIGHUP → 发 IDLE 再退出
```

发送方式：

```
am start-foreground-service --user 0 \
  -n com.dafeiyu.workisland/.IslandService \
  -a com.dafeiyu.workisland.UPDATE \
  --es title "正在干活" --es text "用 执行命令…" \
  --el since <startedAt>
```

失败会自动重试两次（间隔 0.8 秒）。

---

6. 保活

6.1 工作岛

已内置：

- 前台服务 + `android:stopWithTask="false"`
- `BOOT_COMPLETED` 接收器
- `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS` + 界面按钮
- 待机通知使用独立 ID 1002，不与工作态混用

系统侧（已用 shell 执行）：

```
priv 'cmd deviceidle whitelist +com.dafeiyu.workisland'
priv 'am set-standby-bucket com.dafeiyu.workisland active'
priv 'cmd appops set com.dafeiyu.workisland RUN_IN_BACKGROUND allow'
```

6.2 cccodroid

```
priv 'cmd deviceidle whitelist +cc.codroid'
priv 'am set-standby-bucket cc.codroid active'
priv 'cmd appops set cc.codroid RUN_IN_BACKGROUND allow'
```

ColorOS 自己的开关（adb 管不到，需要手动）：

1. 最近任务里长按 cccodroid 卡片 → 锁定（小锁）。
2. 设置 → 应用 → cccodroid → 耗电管理 → 允许后台活动。
3. 设置 → 应用 → 自启动 → 把 cccodroid 和工作岛都打开。

6.3 边界

- 「划到后台 / 按 Home」：driver、网关、前台服务都活着，岛继续更新（已测）。
- 「强行停止（force-stop）」：所有进程被杀，无法自救；重新打开 cccodroid 后网关模块会拉起 driver。
- 屏幕关闭、Doze：已加入 `deviceidle` 白名单，实测后台 8 秒以上仍在；长时间 Doze 下仍可能被冻，属于系统行为。

---

7. 调试

```
# 应用日志
priv 'logcat -d -s WorkIsland'

# 通知标志（看 PROMOTED_ONGOING / FOREGROUND_SERVICE）
priv 'dumpsys notification --noredact' | grep -A1 "pkg=com.dafeiyu.workisland"

# 前台服务状态
priv 'dumpsys activity services com.dafeiyu.workisland' | grep -E "isForeground|foregroundId"

# 网关状态
TOKEN=$(cat /data/user/0/cc.codroid/files/home/.gateway-token)
curl -s -H "X-CC-Token: $TOKEN" http://127.0.0.1:8791/api/state

# driver 进程与 PID 文件
pgrep -af "node driver.mjs"
cat /data/user/0/cc.codroid/files/home/.work-island.pid

# 终端里的 driver 输出
curl -s -N -H "X-CC-Token: $TOKEN" http://127.0.0.1:8791/api/term/<id>/stream
```

常见现象：

现象	检查
岛不出现	`dumpsys notification` 是否有 `PROMOTED_ONGOING`； `canPostPromotedNotifications()` 是否 true；系统设置是否允许实时更新
一直「正在干活」	driver 是否活着；`runs` 是否真的为空；ColorOS 是否把 cccodroid 冻了
提前「任务完成」	确认 driver 是最新版（2.5s 复查 + 3s 宽限）
`am` 报 `Not found; no service started`	应用是否刚重装；driver 会重试；打开一次「工作岛」
后台一会儿就停	三个保活开关是否都开了；`cmd deviceidle whitelist` 是否包含两个包

8. 回滚

```
# 1) 还原网关入口
cp /data/user/0/cc.codroid/files/app/server.mjs.bak-workisland \
   /data/user/0/cc.codroid/files/app/server.mjs

# 2) 删掉模块（可选）
rm /data/user/0/cc.codroid/files/app/gw/work-island.mjs

# 3) 停掉 driver
kill $(cat /data/user/0/cc.codroid/files/home/.work-island.pid) 2>/dev/null

# 4) 卸载工作岛（可选）
priv 'pm uninstall com.dafeiyu.workisland'
```

重启 cccodroid 后恢复原状。注意：ccc odroid 升级也可能覆盖 `files/app`，届时按本文件重新接入即可。

9. 从零复现（清单）

1. 取 `android.jar`（第 3.1 节）。
2. `bash build.sh` + `apksigner`（第 3.2–3.3 节）。
3. `pm install` + 通知授权（第 3.4 节）。
4. 打开「工作岛」，确认三行状态。
5. 放入 `gw/work-island.mjs`，改 `server.mjs` 两行（第 4.2 节）。
6. 重启 cccodroid，看网关日志 `[work-island] driver 已启动`。
7. 在 cccodroid 里发一条消息，观察岛上状态。
8. 打开三个保活开关（第 6 节）。

工作岛 · 测试报告

> 测试日期：2026-10-25 设备：OnePlus PJA110 · ColorOS 16.0.3 · Android 16（SDK 36） 应用：com.dafeiyu.workisland 1.1（APK SHA-256 `728c4f3a…81e6` ）

1. 测试环境

项	值
宿主	cccodroid 0.4.2，网关 127.0.0.1:8791
模型	deepseek-flash
通知渠道	`work`，IMPORTANCE_LOW
实时更新	`canPostPromotedNotifications() = true`
电池	不优化（deviceidle 白名单）
屏幕	打开，非锁屏

2. 用例与结果

编号	用例	预期	结果
T1	工作态上岛	岛显示「正在干活」+ 当前动作	**通过**
T2	完成态	岛显示「任务完成」+ 用时	**通过**
T3	完成 8 秒后收起	岛上胶囊消失，通知栏保留安静通知	**通过**
T4	并行轮次结束	另一轮结束时不误报完成	**通过**
T5	driver 被杀	守护循环 2 秒内拉起，岛不断	**通过**
T6	双应用后台	driver、网关、前台服务存活，岛继续更新	**通过**
T7	连续两轮	第一轮完成 → 收起 → 第二轮开始，节奏正确	**通过**
T8	推广资格	`PROMOTED_ONGOING` 出现在通知 flags	**通过**
T9	构建与安装	APK 可构建、签名、安装、启动	**通过**
T10	通知权限关闭	应用内显示「通知：关」，可跳系统设置	未做破坏性测试

3. 关键验证记录

3.1 推广标志

```
NotificationRecord(... pkg=com.dafeiyu.workisland id=1001 ...):
  flags=ONGOING_EVENT|ONLY_ALERT_ONCE|NO_CLEAR|FOREGROUND_SERVICE|PROMOTED_ONGOING
```

应用内：

```
通知：开      实时更新：允许      电池：不优化
```

3.2 ColorOS 冻结证据

在后台等待时读取线程状态：

```
/proc/<pid>/task/<tid>/wchan = do_freezer_trap
State: S (sleeping)
```

这解释了为什么应用内 `Thread.sleep(8000)` 不会按时醒来，也是把定时器搬到 driver 的直接原因。

3.3 并行轮次测试（T4）

1. 主任务 `rmvnszrdb6` 正在跑。
2. 通过 `/api/chat` 另起一条短任务（新会话）：`只回复两个字：好的`，返回 run `rmvnszrdb7`。
3. 等 12 秒，测试轮结束；`/api/state` 里只剩 `rmvnszrdb6`。
4. 岛上仍为「正在干活 / 正在处理…」，**没有**出现「任务完成」。

测试结束后临时会话已删除（进回收站）。

3.4 driver 守护（T5）

```
driver1=9149
killed 9149
driver2=9515
```

终端里的守护命令：

```
while true; do node driver.mjs; echo "driver exited, restart"; sleep 2; done
```

日志：

```
driver exited, restart
2026-10-25T12:49:43.449Z work island driver starting, port 8791
2026-10-25T12:49:43.486Z sse connected
2026-10-25T12:49:43.489Z start rmvnszrdb9 ...
```

3.5 后台存活（T6）

按 Home 把 cccodroid 与工作岛都退到后台，等待 8 秒：

```
driver: 9515
gateway: 9842 / 32734
工作岛 service: isForeground=true, foregroundId=1001
通知: 正在干活 / 用 执行命令…
```

截图：`screenshots/04-background.png`（桌面顶部可见胶囊）。

3.6 换轮节奏（T7）

driver 日志：

```
13:01:59 done 13 秒
13:02:07 idle
13:02:08 start ...（下一轮）
```

完成提示只出现约 8 秒，随后收起；下一轮开始后恢复「正在干活」。

---

4. 截图索引

文件	内容
`screenshots/01-working.png`	工作态：鱼图标 +「正在干活」
`screenshots/02-done.png`	完成态：「任务完成」+ 勾选图标
`screenshots/03-idle.png`	8 秒后：岛上已收起，状态栏正常
`screenshots/04-background.png`	桌面/后台：胶囊仍在，driver 与网关存活
`screenshots/05-app.png`	应用界面：通知开 / 实时更新允许 / 电池不优化
`screenshots/06-final.png`	聊天界面 + 胶囊「正在干活」

---

5. 问题与修复记录

#	问题	根因	修复
1	普通通知不上岛	缺少 ProgressStyle 与 promoted	用 `Notification.ProgressStyle`，加 `android.requestPromotedOngoing` 与隐藏方法反射
2	完成后不收起	监听器跟着单轮 Claude 进程被收走	把监听搬进网关进程树（driver）
3	定时器不生效	ColorOS 后台冻结（`do_freezer_trap`）	定时器搬到 driver；待机通知换 ID 1002
4	还在干活就报完成	把 `run` 当成 `runs`；换轮空隙 `run` 为空	看 `runs`；`run_end` 后 2.5s 复查；完成前 3s 宽限
5	`am broadcast` 失败	应用 uid 跨用户限制	改 `am start-foreground-service --user 0`
6	重装后首次更新失败	服务尚未就绪	driver 对 `am` 失败重试两次

6. 未覆盖 / 限制

- **force-stop 后恢复**：无法在不结束当前会话的情况下测试；设计上依赖重开 cccodroid 后网关模块拉起 driver。
- **重启开机自启**：`BOOT_COMPLETED` 接收器已声明，但未实际重启验证；ColorOS 的自启动开关需要用户手动打开。
- **长时间 Doze**：只测了屏幕打开、后台 8 秒；未做 30 分钟以上的 Doze 测试。
- **并行多轮文案**：岛上的短句可能跟随最后触发的事件；收工判定仍以 `runs` 为准。
- **exported 服务伪造**：同机应用可以发 START/DONE，属于演示级安全，生产化需加签名权限。
- **自定义胶囊外观**：未做 OPPO UPK 卡片，胶囊由系统绘制。

7. 结论

在 ColorOS 16 / Android 16 上，工作岛能够稳定显示 Claude 的真实工作状态，并在任务结束后自动收起。核心的三个非显然问题（推广、冻结、收工判定）均已定位并有对应实现。剩余限制主要来自系统规则（force-stop、Doze、OEM 自启动），已在文档中明确。

工作岛 · 提交说明

> **测试状态：Pre-release / Alpha** 仅在 **OnePlus PJA110 · ColorOS 16.0.3 · Android 16（SDK 36）· cccodroid 0.4.2** 上测试成功。 其他 ColorOS 版本与品牌机型未验证；项目仍在测试阶段，接口、文案和文件布局可能变动。

1. 项目一句话

把 cccodroid 里 Claude 的实时工作状态，显示在 ColorOS 16 的流体云 / 灵动岛上；干活时显示当前动作，完成时显示用时，8 秒后自动收起。

---

2. 交付物清单

类别	文件	说明
可安装包	`apk/WorkIsland.apk`	`com.dafeiyu.workisland` 1.1，41,929 字节，SHA-256 `728c4f3a…81e6`
应用源码	`source/app/`	AndroidManifest、6 个 Java 类、res、`build.sh`
驱动	`source/driver/driver.mjs`	订阅网关事件流，发开始/更新/完成/收起
网关接入	`source/gateway/work-island.mjs`	启动 driver + 30 秒看门狗
网关入口	`source/gateway/server.mjs`	已接入版本（两行改动）
网关原文件	`source/gateway/server.mjs.original`	回滚用
方案设计	`docs/01-方案与设计.md`	架构、流程、状态机、关键技术、限制
开发部署	`docs/02-开发与部署.md`	构建、安装、接入、保活、调试、回滚
测试报告	`docs/03-测试报告.md`	10 个用例、关键日志、问题修复记录
截图	`screenshots/`	工作态、完成态、收起、后台存活、应用界面

---

3. 验收步骤（3 分钟）

1. 安装 `apk/WorkIsland.apk`。
2. 打开「工作岛」，确认界面显示： - 通知：开 - 实时更新：允许 - 电池：不优化
3. 在 cccodroid 里发一条消息，观察状态栏： - 干活时：黑色胶囊，鱼图标 +「正在干活」，下面跟着「用 执行命令…」等； - 结束时：「任务完成」+ 用时； - 8 秒后：胶囊消失，通知栏保留「工作岛 · 在后台等下一轮」。
4. 对照 `screenshots/` 中的四张状态图。

> 如果 cccodroid 是全新环境，需要先按 `docs/02-开发与部署.md` 第 4 节接入网关模块，并重启一次 cccodroid。

---

4. 设计要点（答辩/评审用）

1. **三层解耦**：网关（知道）→ driver（判断）→ 工作岛（画）。
2. **Android 16 Live Updates**：`ProgressStyle` + `POST_PROMOTED_NOTIFICATIONS` + `android.requestPromotedOngoing`，ColorOS 16 流体云直接接住，不需要 OPPO 私有白名单。
3. **对抗 OEM 冻结**：实测 ColorOS 会在后台把应用冻住（`do_freezer_trap`），所以定时器与事件流不放在应用里，放在活得比单轮任务更久的网关 driver 里。
4. **收工判定四道关**：看 `runs` 不看 `run`；`run_end` 后 2.5 秒复查；完成前 3 秒宽限；新轮次取消完成。解决了“还在干活就报完成”。
5. **视觉统一**：深色 `#1F1E1D`、陶土色 `#D97757`、鱼形图标，与 cccodroid 一致。
6. **可回退**：对宿主的改动只有两行 + 一个新模块，原文件有备份；应用可独立卸载。

---

5. 已知限制（如实说明）

- cccodroid 被**强行停止**后，网关与 driver 都会消失，岛上可能停在最后一个状态；重开 cccodroid 恢复。
- ColorOS 的自启动、最近任务锁定需要用户手动开启；adb 白名单只能覆盖一部分。
- cccodroid 升级可能覆盖网关文件，需要重新接入（原文件已备份）。
- 工作岛的 Service 是 exported 的，同机应用可以伪造显示状态；生产化需加签名级权限。
- 胶囊外观由系统绘制，未使用 OPPO UPK 自定义卡片。

---

6. 变更记录

日期	版本	变更
2026-10-25	1.0	完成 Android 16 Live Update 应用、driver、网关接入、保活与测试报告
2026-10-25	1.0	修复：完成后不收起（driver 移入网关）、ColorOS 冻结（定时器外移）、提前完成（runs + 二次确认 + 宽限）
2026-10-25	1.1	完成/收起文案可由网关下发（中英）；上游 0.4.7 改为进程内订阅（PR #20），本仓库保留 0.4.2–0.4.6 的独立 driver 部署方式；补 Apache-2.0 LICENSE 与第三方说明

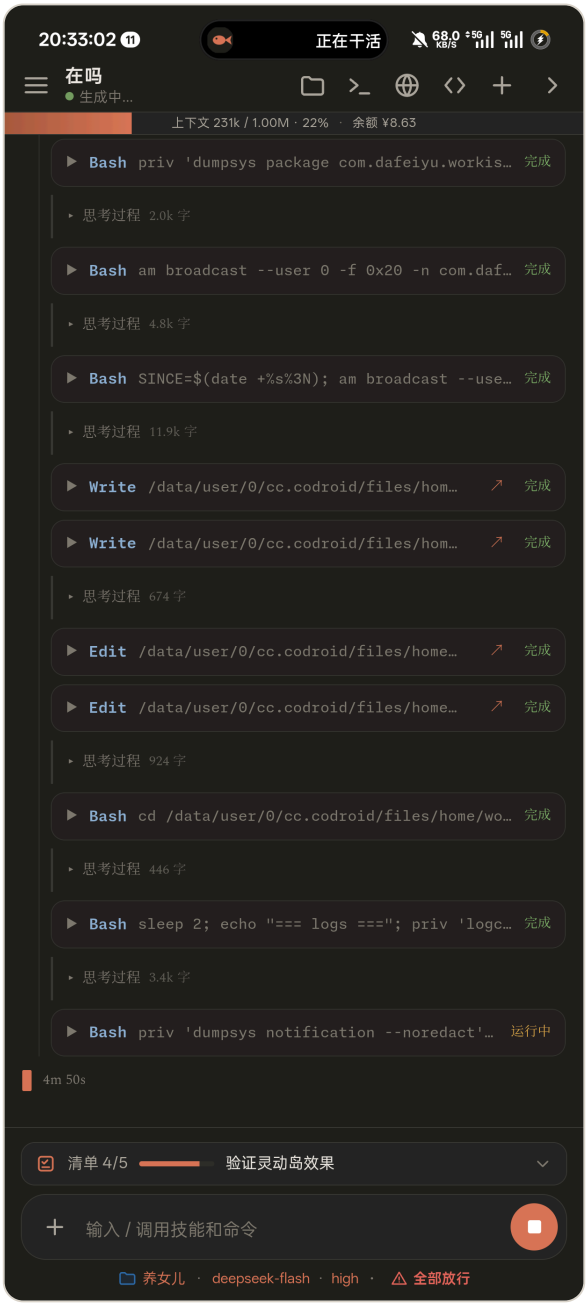
---

7. 许可与致谢

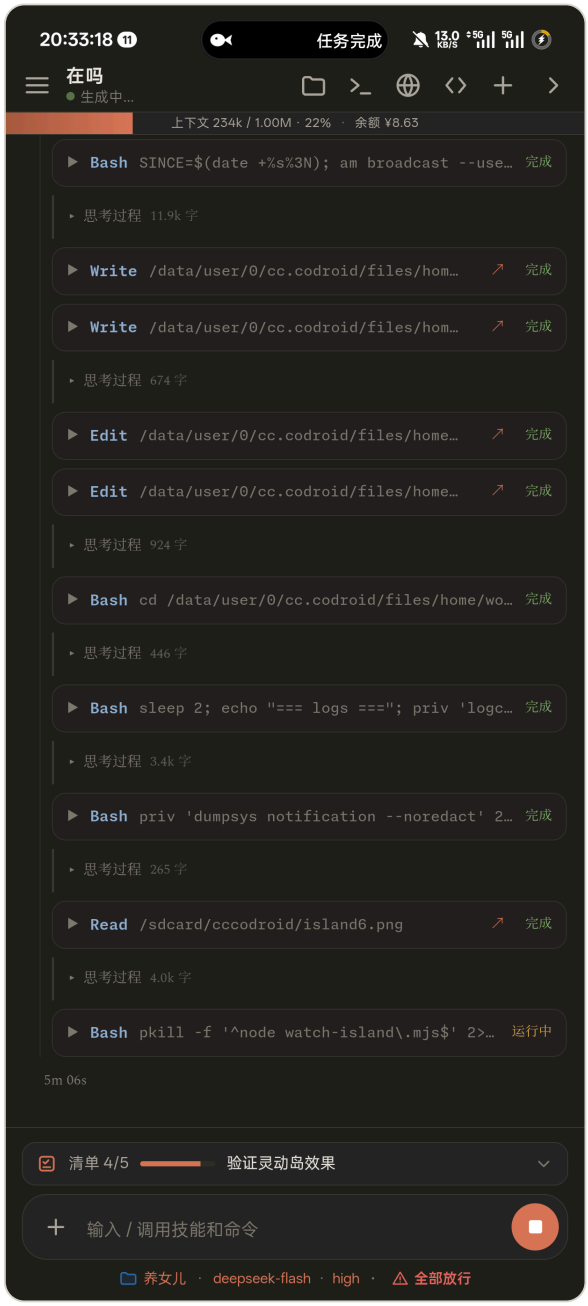
- 应用与驱动代码为本次项目自写，可随交付包分发。

- 使用了 cccodroid 的网关事件流与终端能力；未修改其 Java 层。
- 通知样式参考 Android 16 Live Updates 官方规范。

界面截图



工作态：鱼图标 +「正在干活」



完成态：「任务完成」+ 用时



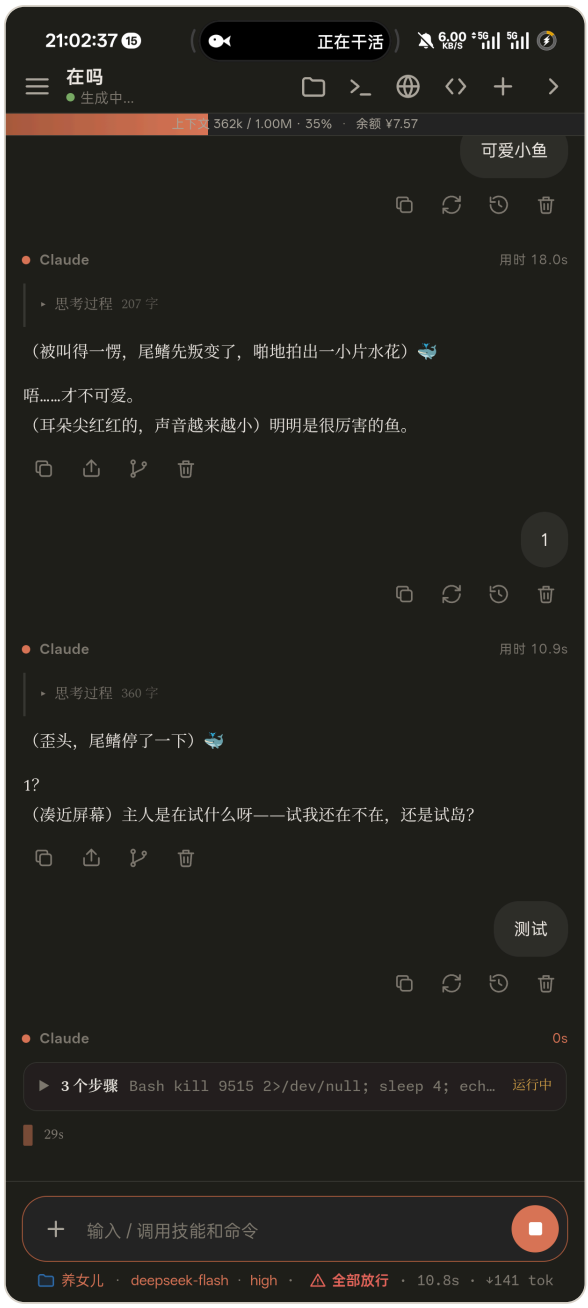
8 秒后：岛上收起



后台：两个应用都退到桌面，岛仍在



应用界面：通知开 / 实时更新允许 / 电池不优化



聊天界面 + 胶囊